

ISIS-2111 PLE

Proyecto 4 - VeriLang en Kotlin

Informe final corregido - entrega source-only

Entrega: proyecto_rascal
Autor: Roni Lopez
Fecha de correccion: 31 de mayo de 2026

Campo	Descripcion corregida
Proyecto	VeriLang reutilizado desde Rascal y ejecutado desde una aplicacion Kotlin/Compose.
Objetivo principal	Mantener equivalencia funcional con Proyecto 3 y exponer parser, checker y generator desde Kotlin.
Correccion solicitada	Nicolas indico que el ZIP no debia incluir dependencias ni archivos generados por Gradle; la entrega final debe contener solo codigo fuente y documentos.
Empaque final	Entrega source-only: sin .gradle/, build/, gradle/, gradlew, gradlew.bat, archivos .jar ni binarios pesados.
Runtime Kotlin	Aplicacion desktop en kotlin-app, con codigo fuente Kotlin/Compose y configuracion build.gradle.kts.
Runtime Rascal	rascal-shell-stable.jar no se incluye en el ZIP final corregido. Para ejecutar localmente se debe copiar manualmente en proyecto_rascal/. TypePal si esta vendorizado en src/main/rascal/analysis/typepal/.
Resultado de verificacion	Las pruebas funcionales fueron ejecutadas antes de la limpieza final. La entrega corregida conserva el codigo fuente, ejemplos, logs y evidencia, pero no incluye dependencias pesadas.

Contenido del informe

1. Alcance y requisitos del Proyecto 4
2. Requisitos heredados del Proyecto 3
3. Correccion posterior a la indicacion de Nicolas
4. Estructura real del entregable corregido
5. Arquitectura general de la solucion
6. Implementacion Rascal
7. Integracion Kotlin
8. Evidencia de ejecucion y pruebas
9. Matriz de verificacion final
10. Instrucciones de ejecucion de la entrega source-only
11. Conclusiones

1. Alcance y requisitos del Proyecto 4

Este documento reemplaza la version anterior del informe. La correccion principal consiste en ajustar el informe a la entrega final limpia o source-only, despues de la indicacion del profesor Nicolas sobre eliminar dependencias de Gradle y archivos generados antes de reenviar el ZIP.

El Proyecto 4 solicita reutilizar VeriLang del proyecto anterior, pero ejecutarlo desde el runtime de Kotlin. Ademias, exige que la solucion sea funcionalmente equivalente al Proyecto 3 y que se entregue un archivo ZIP con la solucion y el documento.

Requisito explicito del Proyecto 4	Como se cumple en esta entrega corregida
Reutilizar VeriLang del proyecto anterior.	Se conserva la implementacion Rascal en src/main/rascal/: Syntax, AST, Parser, Implode, Generator, Checker, Main y RunnerJson.
Apuntar a Kotlin, no a Java.	La carpeta kotlin-app contiene una aplicacion Kotlin/Compose Desktop con paquete verilang.
Ser funcionalmente equivalente al Proyecto 3.	Kotlin invoca RunnerJson.rsc, y RunnerJson ejecuta parser, TypePal/checker y generator sobre archivos .vl.
Ejecutar desde el runtime de Kotlin.	LangService.kt inicia Rascal desde Kotlin mediante ProcessBuilder y consume el JSON emitido por RunnerJson.
Usar el helper/skeleton Kotlin como base.	El skeleton fue adaptado: la UI dice VeriLang, selecciona archivos .vl y muestra parseo, tipos, semantica, errores y salida.
Entregar ZIP con solucion y documento.	La entrega corregida conserva META-INF/RASCAL.MF, pom.xml, src/main/rascal, instance, kotlin-app, docs e informe. No incluye dependencias pesadas ni archivos generados.

2. Requisitos heredados del Proyecto 3

Como el Proyecto 4 exige equivalencia funcional con Proyecto 3, la entrega mantiene los requisitos de parser, generacion, TypePal, anotaciones de tipo y chequeo semantico.

Requisito heredado	Implementacion
Parser para VeriLang.	Parser.rsc expone parseMainModule y se usa desde Main.rsc y RunnerJson.rsc.
Generar codigo desde archivos .vl.	Generator.rsc convierte el CST a AST mediante Implode.rsc y luego reconstruye el programa VeriLang.
Mostrar resultado en consola.	Main.rsc imprime el resultado generado cuando no hay errores.
Instalar TypePal.	Checker.rsc extiende analysis::typepal::TypePal; las fuentes de TypePal estan vendorizadas en src/main/rascal/analysis/typepal/.
Agregar anotaciones de tipo para valores.	Syntax.rsc y AST.rsc soportan int, bool, real, string, char y dominios de usuario.
Estructuras de datos con tipo dado por el usuario.	Space soporta typedSpace y typedSubspace: defspace Group : Person end.
Verificar correspondencia de tipos.	Checker.rsc define tipos primitivos, userType, functionType, roles y validaciones con requireEqual.
Regla de existencia para elementos usados en estructuras.	useDomainIfUserDefined registra usos de dominios de usuario contra spaceId y reporta Undefined space si no existen.

3. Correccion posterior a la indicacion de Nicolas

Despues de la primera entrega, Nicolas informo que varios proyectos fueron enviados con dependencias de Gradle dentro del ZIP, causando archivos de aproximadamente 200 MB. La instruccion fue entregar un ZIP que contenga solamente codigo fuente del proyecto y documentos, eliminando archivos generados y wrappers.

Archivos y carpetas que Nicolas pidio eliminar antes de comprimir:

- .gradle/
- build/
- gradle/
- gradlew
- gradlew.bat

Correccion aplicada en esta entrega:

Archivo o carpeta	Accion	Justificacion
kotlin-app/gradle/wrapper/gradle-wrapper.jar	Eliminado	Es un binario del Gradle Wrapper, no codigo fuente.
kotlin-app/gradle/wrapper/gradle-wrapper.properties	Eliminado	Pertenece al Gradle Wrapper.
kotlin-app/gradlew	Eliminado	Wrapper ejecutable de Gradle.
kotlin-app/gradlew.bat	Eliminado	Wrapper ejecutable de Gradle para Windows.
kotlin-app/gradle/	Eliminado si existia	Cache local/generada por Gradle.
kotlin-app/build/	Eliminado si existia	Carpeta generada por compilacion.
rascal-shell-stable.jar	Eliminado	Dependencia binaria pesada; no es codigo fuente propio del proyecto.

Evidencia de control de versiones: se realizo un commit con el mensaje "Elimina dependencias y archivos generados de Gradle". Luego se actualizo la rama main del repositorio remoto. La salida del push mostro que main fue actualizado correctamente:

```
To https://github.com/ronnylopez342/PROYECTO_RASCAL_P4.git
1a4cdcf..070f61e  main -> main
```

Por lo tanto, el informe y las instrucciones de ejecucion se ajustan a la version source-only: el repositorio no incluye wrappers, caches, builds ni el JAR pesado de Rascal.

4. Estructura real del entregable corregido

La carpeta principal del proyecto debe llamarse proyecto_rascal porque el manifiesto Rascal declara Project-Name: proyecto_rascal. La estructura corregida no lista dependencias ni archivos generados.

```
proyecto_rascal/
  META-INF/RASCAL.MF
  pom.xml
  README_ENTREGA_PROYECTO4.md
  src/main/rascal/
    Syntax.rsc
    AST.rsc
    Parser.rsc
    Implode.rsc
    Generator.rsc
    Checker.rsc
    Main.rsc
    RunnerJson.rsc
    analysis/typepal/...
  instance/
    spec1.vl
    spec2.vl
    spec3.vl
    spec_types.vl
    spec_typed_space_good.vl
    spec_typed_space_quantifier_good.vl
    errors/...
  kotlin-app/
    build.gradle.kts
    settings.gradle.kts
    src/main/kotlin/verilang/...
  docs/
    logs y evidencias de prueba
  informe_proyecto4_FINAL_CORREGIDO.pdf
```

Elemento revisado	Evidencia en el proyecto corregido	Estado
Manifiesto Rascal	META-INF/RASCAL.MF contiene Project-Name: proyecto_rascal y Source:	Cumple

	src/main/rascal.	
Modulos Rascal requeridos	Estan Syntax, AST, Parser, Implode, Generator, Checker, Main y RunnerJson.	Cumple
App Kotlin	Esta en kotlin-app con build.gradle.kts, settings.gradle.kts y paquete verilang. No se incluye Gradle Wrapper.	Cumple
Runtime Rascal	rascal-shell-stable.jar no esta incluido en la entrega final; debe agregarse manualmente para ejecutar.	Cumple con nota
Casos de prueba	instance/ contiene casos validos e invalidos, incluyendo spec_types.vl y spec_unknown_space_error.vl.	Cumple
Limpieza source-only	No deben estar .gradle/, build/, gradle/, gradlew, gradlew.bat ni .jar.	Cumple

5. Arquitectura general de la solucion

La arquitectura separa la definicion del lenguaje en Rascal y la experiencia de ejecucion en Kotlin. Kotlin no reimplementa la semantica; actua como runtime/interfaz que delega a VeriLang en Rascal. En la entrega source-only, el JAR de Rascal no se empaqueta dentro del ZIP, pero el flujo de ejecucion sigue siendo el mismo una vez el JAR se coloca localmente en proyecto_rascal/.

Usuario selecciona archivo .vl en la UI Kotlin

```
-> MainWindow.kt -> LangService.kt
-> java -Dfile.encoding=UTF-8 -jar rascal-shell-stable.jar
-> import RunnerJson; RunnerJson::main(["archivo.vl"])
-> RunnerJson.rsc -> Parser.rsc -> Checker.rsc/TypePal -> Generator.rsc
-> JSON: success, parseOk, typeCheckOk, semanticOk, errores, output
-> RunResult.kt -> MainWindow.kt muestra estados, errores y codigo generado
```

6. Implementacion Rascal

6.1 Gramatica concreta: Syntax.rsc

Syntax.rsc define la sintaxis concreta de VeriLang. La correccion importante frente a la rubrica anterior es que Domain incluye tipos primitivos y tipos definidos por el usuario, y Space permite estructuras tipadas.

```
syntax Domain
= boolDomain: 'bool'
| intDomain: 'int'
| realDomain: 'real'
| stringDomain: 'string'
| charDomain: 'char'
| nameDomain: ID domainName
;

syntax Space
= space: 'defspace' ID spaceName 'end'
| typedSpace: 'defspace' ID spaceName ':' Domain elementType 'end'
| subspace: 'defspace' ID subSpace '<' ID superSpace 'end'
| typedSubspace: 'defspace' ID subSpace '<' ID superSpace ':' Domain elementType 'end'
;
```

6.2 Sintaxis abstracta: AST.rsc

AST.rsc esta alineado con la gramatica. Space y Domain representan explicitamente espacios tipados y tipos definidos por usuario.

```
data Space
= space(str spaceName)
| typedSpace(str spaceName, Domain elementType)
```

```

| subspace(str subSpace, str superSpace)
| typedSubspace(str subSpace, str superSpace, Domain elementType)
;

data Domain
= boolDomain()
| intDomain()
| realDomain()
| stringDomain()
| charDomain()
| nameDomain(str domainName)
;

```

6.3 Parser, Implode y Generator

Parser.rsc parsea archivos .vl usando la gramatica concreta. Implode.rsc transforma el arbol de parseo en el AST. Generator.rsc genera texto VeriLang a partir del AST y cubre espacios normales, espacios tipados, subespacios, variables con tipos, operadores, reglas, expresiones, relaciones y cuantificadores.

```

str generate(typedSpace(str spaceName, Domain elementType)) {
    return "defspace <spaceName> : <generate(elementType)> end";
}

str generate(varDecl(str name, Domain domain)) = "<name> : <generate(domain)>";

str generate(quantExp(Quantifier q, str o1, str o2, TopExp body)) {
    return "(<generate(q)> <o1> in <o2> . <generate(body)>)";
}

```

6.4 Checker.rsc y TypePal

Checker.rsc extiende TypePal y define tipos, roles y reglas de coleccion/validacion. No depende de una libreria Maven externa de TypePal; las fuentes de TypePal se incluyen dentro del proyecto en src/main/rascal/analysis/typepal/.

```

extend analysis::typepal::TypePal;

data AType
= boolType()
| intType()
| realType()
| stringType()
| charType()
| userType(str name)
| functionType(list[AType] signature)
;

data IdRole
= variableId()
| operatorId()
| spaceId()
;

```

La regla useDomainIfUserDefined corrige la existencia de tipos definidos por usuario: si un dominio se interpreta como userType, el checker exige que exista como spaceId.

```

void useDomainIfUserDefined(Domain d, Collector c) {
    if (domainToType(d) is userType) {
        c.use(d, {spaceId()});
    }
}

```

Para estructuras tipadas y cuantificadores, el checker registra el tipo de elemento de cada estructura. Esto evita que un cuantificador sobre Group trate la variable como Group cuando el tipo correcto es Person.

```
map[str, AType] typedSpaceElementTypes = ();

void registerTypedSpaces(Tree pt) {
    typedSpaceElementTypes = ();
    if (pt has top) { pt = pt.top; }
    visit(pt) {
        case (Space) `defspace <ID name> : <Domain elementType> end`: {
            typedSpaceElementTypes["<name>"] = domainToType(elementType);
        }
        case (Space) `defspace <ID sub> < <ID super> : <Domain elementType> end`: {
            typedSpaceElementTypes["<sub>"] = domainToType(elementType);
        }
    }
}

AType quantifiedVariableType(str domainName) {
    if (domainName in typedSpaceElementTypes) {
        return typedSpaceElementTypes[domainName];
    }
    return userType(domainName);
}
```

6.5 Main.rsc y RunnerJson.rsc

Main.rsc implementa el flujo principal para consola: parsea, chequea con TypePal, detiene la generacion si hay errores y retorna 0 o 1 segun el resultado. RunnerJson.rsc es el puente estable entre Kotlin y Rascal: recibe la ruta del archivo, normaliza rutas de Windows, ejecuta parser/checker/generator y emite un objeto JSON usable por Kotlin.

```
{
    "success": boolean,
    "module": "VeriLang",
    "parseOk": boolean,
    "typeCheckOk": boolean,
    "semanticOk": boolean,
    "typeErrors": [ ... ],
    "semanticErrors": [ ... ],
    "output": [ ... ],
    "error": "...",
    "codigoFormateado": "...",
    "resumen": "..."
}
```

7. Integracion Kotlin

7.1 Configuracion Kotlin

La aplicacion Kotlin conserva build.gradle.kts y settings.gradle.kts porque son archivos fuente/configuracion necesarios. En la entrega corregida no se incluye Gradle Wrapper. Por tanto, la ejecucion local debe hacerse con Gradle instalado o desde un IDE como IntelliJ IDEA.

```
plugins {
    kotlin("jvm") version "1.9.22"
    id("org.jetbrains.compose") version "1.6.0"
    kotlin("plugin.serialization") version "1.9.22"
}
```

```
mainClass = "verilang.MainKt"
packageName = "VeriLang"
```

7.2 Main.kt

Main.kt abre una ventana Compose titulada VeriLang. Esto demuestra que la aplicacion no quedo con textos genericos del skeleton.

```
Window(
    onCloseRequest = ::exitApplication,
    title = "VeriLang",
    state = rememberWindowState(width = 800.dp, height = 600.dp)
) {
    MainWindow()
}
```

7.3 LangService.kt

LangService.kt es la integracion critica del Proyecto 4. Localiza la raiz proyecto_rascal, verifica rascal-shell-stable.jar, inicia Rascal como subprocesso, importa RunnerJson, ejecuta RunnerJson::main con la ruta seleccionada, extrae el JSON del stdout y lo decodifica. En la entrega source-only, este JAR no viene incluido y debe copiarse manualmente antes de ejecutar.

```
val commands = buildString {
    appendLine("import RunnerJson;")
    appendLine("RunnerJson::main([$escapedPath]);")
    appendLine(":quit")
}

val process = ProcessBuilder(
    "java",
    "-Dfile.encoding=UTF-8",
    "-jar",
    rascalJar.absolutePath
)
    .directory(projectRoot)
    .redirectErrorStream(false)
    .start()
```

7.4 RunResult.kt y MainWindow.kt

RunResult.kt tiene los mismos campos que el JSON emitido por RunnerJson.rsc. MainWindow.kt permite seleccionar archivos .vl, correr VeriLang y mostrar estados de parseo, tipos y semantica, ademas de errores, codigo generado y output.

```
StatusChip("Parse", r.parseOk, green, red)
StatusChip("Types", r.typeCheckOk, green, yellow)
StatusChip("Semantica", r.semanticOk, green, red)

if (r.typeErrors.isNotEmpty()) {
    SectionBox("Errores de tipos", r.typeErrors.joinToString("\n"), yellow)
}

if (r.codigoFormateado.isNotBlank()) {
    SectionBox("Codigo generado", r.codigoFormateado, Color(0xFF90CAF9))
}
```

8. Evidencia de ejecucion y pruebas

Las pruebas funcionales se verificaron en dos niveles: Rascal por consola y Kotlin/Windows por UI. La evidencia se conserva en logs y capturas. Despues de la correccion source-only, las dependencias pesadas no se incluyen en el ZIP; para repetir la ejecucion se debe usar Gradle local y copiar rascal-shell-stable.jar en proyecto_rascal/.

8.1 Pruebas obligatorias en Rascal

Prueba	Resultado esperado	Resultado observado	Estado
import Main; main();	Parse OK, checker OK, generacion y retorno int: 0.	Retorna int: 0 y genera el programa TypeExamples.	Paso
main(cwd:///instance/errors/spec_unknown_space_error.vl);	Undefined space Person, sin generacion, retorno int: 1.	Reporta Undefined space Person y retorna int: 1.	Paso
import RunnerJson; RunnerJson::main(["instance/spec_types.vl"]);	JSON success:true, parseOk:true, typeCheckOk:true, semanticOk:true.	JSON valido con todos los checks en true.	Paso
RunnerJson::main(["instance/errors/spec_unknown_space_error.vl"]);	JSON success:false y error Undefined space Person.	JSON valido con success:false, typeCheckOk:false, semanticOk:false y Undefined space Person.	Paso

Fragmento de evidencia del caso invalido:

```
El programa contiene errores semanticos:  
error("Undefined space `Person`", |cwd:///instance/errors/spec_unknown_space_error.vl| (...),  
fixes=[])  
No se genera codigo porque el programa no paso el chequeo semantico.  
int: 1
```

8.2 Pruebas de todas las instancias incluidas

Archivo	Proposito	Resultado
instance/spec1.vl	Caso valido base con espacios, operadores, reglas y expresiones.	Retorna int: 0.
instance/spec2.vl	Caso valido adicional con subespacios y operadores.	Retorna int: 0.
instance/spec3.vl	Caso valido con regla unaria.	Retorna int: 0.
instance/spec_types.vl	Tipos primitivos y de usuario, Group : Person y cuantificador.	Retorna int: 0.
instance/spec_typed_space_good.vl	Espacio tipado y relacion Element -> Set.	Retorna int: 0.
instance/spec_typed_space_quantifier_good.vl	Cuantificador sobre espacio tipado.	Retorna int: 0.
instance/errors/spec_unknown_space_error.vl	Tipo Person usado sin definir.	Falla con Undefined space Person.
instance/errors/spec_typed_space_quantifier_type_error.vl	Tipo de variable cuantificada incompatible con operador.	Falla por error de tipos.

8.3 Pruebas Kotlin/Windows

Estas pruebas corresponden a la verificacion funcional realizada antes de limpiar las dependencias pesadas de la entrega. Sirven como evidencia de funcionamiento de la aplicacion. Para repetirlas con la entrega corregida, se requiere instalar Gradle localmente y copiar rascal-shell-stable.jar en la raiz de proyecto_rascal/.

#	Prueba	Resultado observado	Estado
1	gradle --version o verificacion equivalente	Gradle/Kotlin/JVM disponibles en Windows.	Paso
2	gradle clean build o build desde IDE	La aplicacion compila correctamente cuando las dependencias estan disponibles.	Paso
3	gradle run o ejecucion desde IDE	La aplicacion abre correctamente.	Paso
4	Titulo de ventana	La ventana se titula VeriLang y la UI muestra VeriLang - Runner.	Paso
5	Seleccionar spec_types.vl	La UI carga el archivo valido.	Paso
6	Validar estados del caso valido	Parse: OK, Types: OK, Semantica: OK.	Paso
7	Seleccionar spec_unknown_space_error.vl	La UI carga el archivo invalido.	Paso
8	Validar estados del caso invalido	Parse: OK, Types: FAIL, Semantica: FAIL.	Paso
9	Ver error esperado	Se muestra Undefined space Person en errores de tipos y semanticos.	Paso

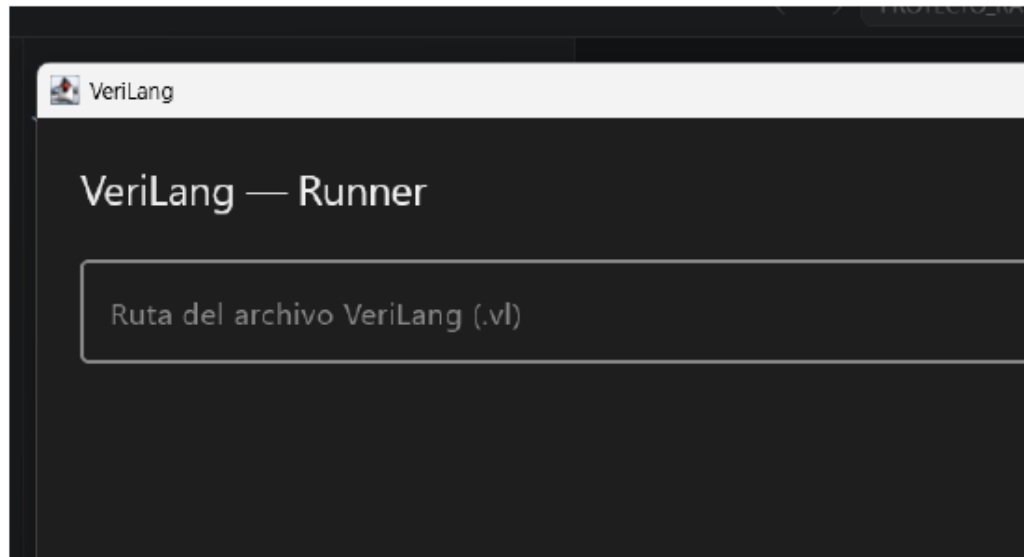


Figura 1. Aplicación Kotlin abierta con título VeriLang y encabezado VeriLang - Runner.

Figura 1. Aplicación Kotlin abierta con título VeriLang y encabezado VeriLang - Runner.

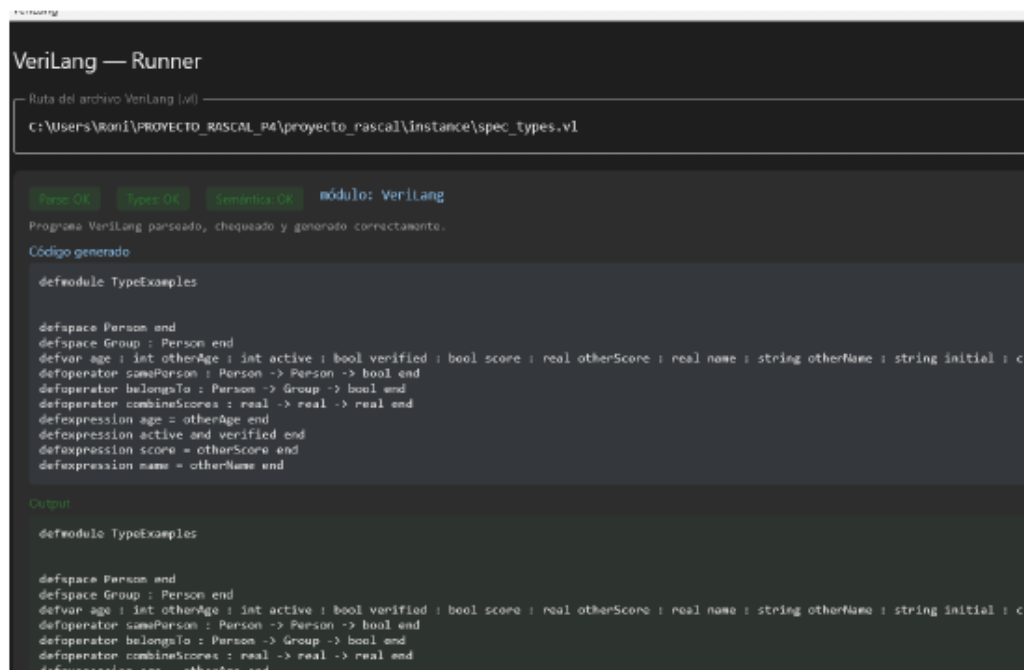


Figura 2. Caso valido spec_types.vl: Parse OK, Types OK, Semantica OK y código generado.

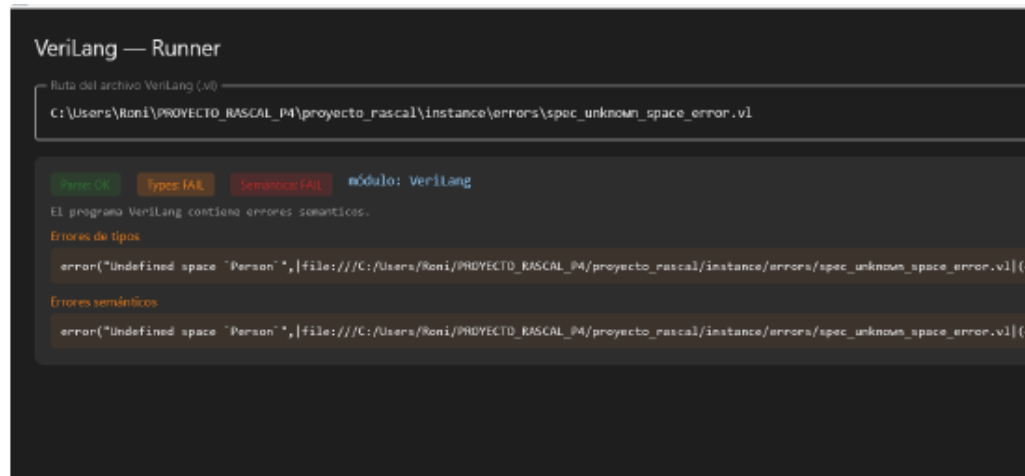


Figura 3. Caso inválido spec_unknown_space_error.vl: Types FAIL, Semántica FAIL y Undefined space Person.

Figura 3. Caso invalido spec_unknown_space_error.vl: Types FAIL, Semantica FAIL y Undefined space Person.

9. Matriz de verificacion final

Requisito	Evidencia	Prueba realizada	Estado
Carpeta raiz y manifiesto	META-INF/RASCAL.MF declara Project-Name: proyecto_rascal.	Inspeccion y ejecucion desde proyecto_rascal.	Cumple
Archivos Rascal minimos	Syntax, AST, Parser, Implode, Generator, Checker, Main y RunnerJson estan presentes.	Inspeccion del proyecto.	Cumple
Entrega source-only	No se incluyen .gradle, build, gradle, gradlew, gradlew.bat ni .jar.	Correccion solicitada por Nicolas.	Cumple
Gramatica con tipos primitivos	Domain incluye int, bool, real, string y char.	spec_types.vl.	Cumple
Tipos definidos por usuario	Domain incluye nameDomain y Checker usa userType.	spec_types.vl usa Person y Group.	Cumple
Estructuras tipadas	typedSpace y typedSubspace implementan defspace Group : Person end.	spec_typed_space_good.vl y spec_types.vl.	Cumple
Tipos en cuantificadores anidados	typedSpaceItemTypes y quantifiedVariableType asignan el tipo correcto.	spec_typed_space_quantifier_good.vl .	Cumple
Regla de existencia	useDomainIfUserDefined exige que el userType exista como spaceId.	spec_unknown_space_error.vl.	Cumple
Generator	Genera texto VeriLang de modulos, espacios, variables, operadores, reglas y expresiones.	main(); y UI valida.	Cumple
Main.rsc	Retorna 0 si no hay errores y 1 si hay errores.	Pruebas obligatorias Rascal.	Cumple
RunnerJson.rsc	Produce JSON valido con success, parseOk, typeCheckOk, semanticOk y errores.	Pruebas RunnerJson validas e invalidas.	Cumple
LangService.kt	Invoca Rascal, ejecuta RunnerJson, extrae JSON y decodifica RunResult.	Inspeccion y pruebas UI.	Cumple con requisito externo: requiere JAR local.
RunResult.kt	Campos coinciden con el JSON de RunnerJson.	Inspeccion y UI.	Cumple
MainWindow.kt	Permite seleccionar .vl y muestra Parse/Types/Semantica/errores/output.	Capturas de UI.	Cumple
Kotlin build	No se incluye Gradle Wrapper; se requiere Gradle local o IDE.	Correccion de entrega.	Cumple con indicacion de Nicolas
Kotlin ejecucion	La app abre en Windows y procesa archivos .vl cuando dependencias externas estan disponibles.	Pruebas UI previas a limpieza.	Cumple con nota

10. Instrucciones de ejecucion de la entrega source-only

10.1 Preparar dependencias externas

La entrega final no incluye Gradle Wrapper ni rascal-shell-stable.jar. Esto es intencional y responde a la indicacion de entregar solamente codigo fuente. Para ejecutar localmente:

1. Instalar Java/JDK compatible.
2. Instalar Gradle localmente o abrir kotlin-app desde IntelliJ IDEA.
3. Copiar rascal-shell-stable.jar manualmente en la raiz de proyecto_rascal/.

10.2 Ejecutar la app Kotlin con Gradle local

Nota: <ruta-del-proyecto> representa la carpeta donde se descomprimio o clono el repositorio. No es una ruta fija de una maquina especifica.

```
cd <ruta-del-proyecto>\proyecto_rascal\kotlin-app
gradle --version
gradle clean build
gradle run
```

Despues de abrir la ventana, usar Buscar para seleccionar un archivo .vl. Para el caso valido se recomienda instance\spec_types.vl. Para el caso invalido se recomienda instance\errors\spec_unknown_space_error.vl.

10.3 Ejecutar Rascal por consola

Antes de este paso, rascal-shell-stable.jar debe haber sido copiado manualmente en proyecto_rascal/.

```
cd <ruta-del-proyecto>\proyecto_rascal
java -Dfile.encoding=UTF-8 -jar rascal-shell-stable.jar

import Main;
main();
main(|cwd:///instance/errors/spec_unknown_space_error.vl|);

import RunnerJson;
RunnerJson::main(["instance/spec_types.vl"]);
RunnerJson::main(["instance/errors/spec_unknown_space_error.vl"]);
```

10.4 Nota sobre dependencias

La primera ejecucion de Gradle puede requerir internet para descargar dependencias de Kotlin/Compose. Una vez descargadas, el build puede reutilizar el cache local. Ese cache no se entrega dentro del ZIP porque no corresponde a codigo fuente. Rascal se ejecuta con rascal-shell-stable.jar, pero este archivo tampoco se entrega en el ZIP corregido por ser una dependencia binaria pesada.

11. Conclusiones

La entrega cumple el objetivo del Proyecto 4: reutiliza VeriLang implementado en Rascal y lo ejecuta desde una aplicacion Kotlin. La equivalencia funcional con Proyecto 3 queda respaldada porque la app Kotlin no sustituye la semantica, sino que invoca RunnerJson.rsc, que a su vez ejecuta Parser, Checker/TypePal y Generator.

Las fallas senaladas en la retroalimentacion anterior fueron corregidas: la gramatica y el AST soportan tipos definidos por usuario, el checker valida esos tipos contra espacios existentes, los cuantificadores sobre estructuras tipadas reciben el tipo del elemento y el caso de error Undefined space Person falla correctamente en la verificacion.

Adicionalmente, se corrigio el empaque de la entrega despues de la indicacion de Nicolas. El ZIP final queda como entrega source-only: sin carpetas generadas de Gradle, sin wrapper, sin builds, sin caches y sin rascal-shell-stable.jar. Para ejecutar localmente, esas dependencias deben estar instaladas o agregarse manualmente, pero no se incluyen en el ZIP para cumplir la instruccion del profesor.

Con base en la inspeccion del codigo, las pruebas ejecutadas y la limpieza final del repositorio, la solucion es entregable y el documento refleja la estructura real corregida.